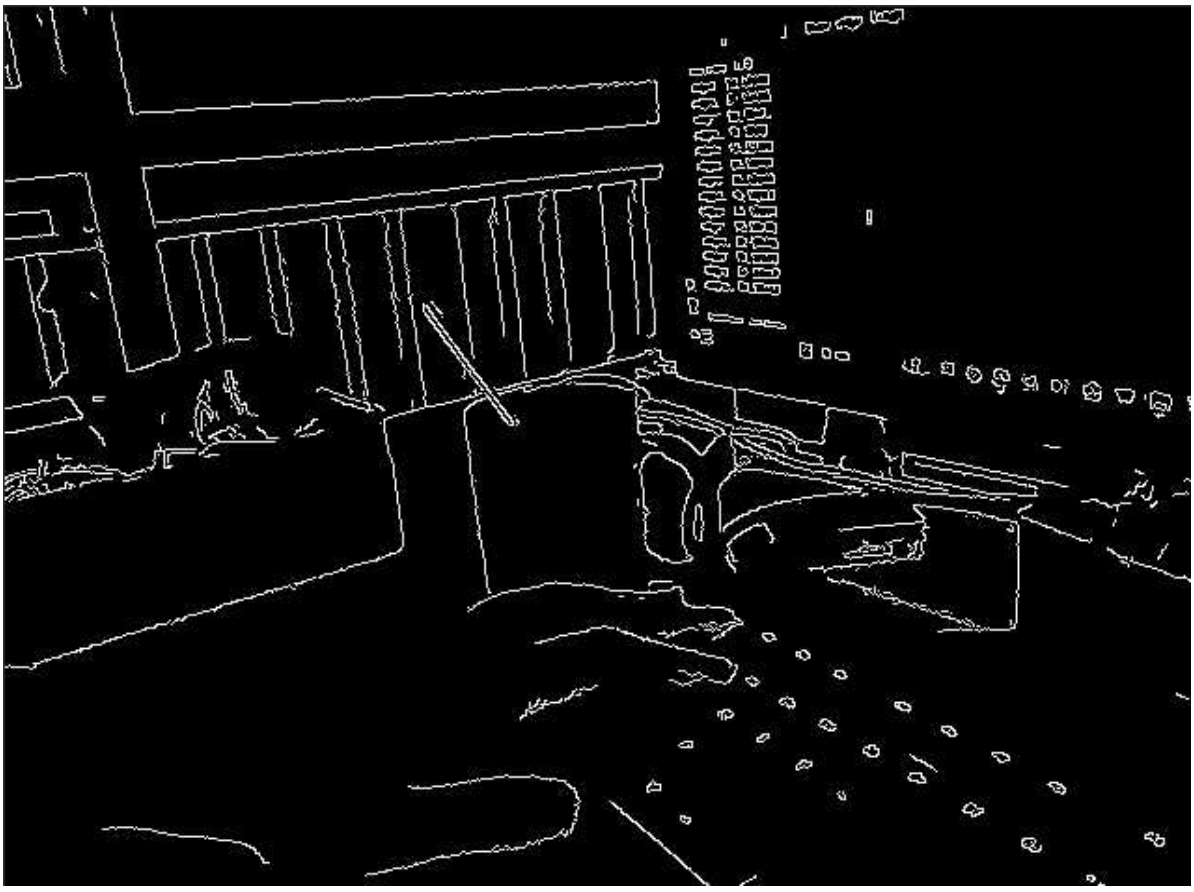
边缘检测-灰度图

边缘检测-灰度图
- 例程效果
- 代码概述
  - 导入模块
  - 设置图像尺寸
    - 初始化摄像头
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置边缘检测参数
    - 图像处理与边缘检测
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/7.grayscale_find_edges.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现灰度图边缘检测（Canny 算法）。



代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import _thread
import cv_lite                # cv_lite 扩展模块（包含边缘检测）
import ulab.numpy as np       # NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头

```
sensor = Sensor(id=2, width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.GRAYSCALE)  # 设置为灰度图像输出 / Grayscale mode
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。
- 将输出帧大小调整为 640x480，并设置为灰度图模式（单通道图像，节省内存和计算资源，适合边缘检测）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
             to_ide=True, quality=50)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置边缘检测参数

```
threshold1 = 50  # 低阈值 / Lower threshold
threshold2 = 80  # 高阈值 / Upper threshold
clock = time.clock()  # 启动帧率计时器 / Start FPS timer
```

- `threshold1`：Canny 边缘检测的低阈值，用于控制弱边缘的检测。
- `threshold2`：Canny 边缘检测的高阈值，用于控制强边缘的检测，值越大检测到的边缘越少。
- `clock`：用于计算帧率。

图像处理与边缘检测

```
while True:
    clock.tick()

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 ndarray 引用 / Get ndarray reference

    # 调用 cv_lite 扩展执行边缘检测 / Perform edge detection
    # 返回灰度边缘图像 ndarray / Returns edge image ndarray
    edge_np = cv_lite.grayscale_find_edges(
        image_shape, img_np, threshold1, threshold2)

    # 包装边缘图像用于显示 / Wrap ndarray as image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                          alloc=image.ALLOC_REF, data=edge_np)

    # 显示边缘图像 / Show edge image
    Display.show_image(img_out)

    # 清理内存并打印帧率 / Cleanup and print FPS
    gc.collect()
    print("edges:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **边缘检测**：调用 `cv_lite.grayscale_find_edges()` 函数进行 Canny 边缘检测，返回一个灰度边缘图像的 NumPy 数组。
- **图像包装和显示**：将边缘检测结果包装为图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `threshold1`：低阈值，值越小能检测到更多弱边缘，但可能引入噪声；建议从 50 开始调整。
- `threshold2`：高阈值，值越大检测到的强边缘越少，适合噪声较多的环境；增加此值可减少假边缘。